

Контроль доступа и роли

ЗАНЯТИЕ 13

ИНСТРУКТОР: АНДРЕЙ ПОМЕЛОВ

Access Control Logic (Roles) — управление правами пользователей





Что такое авторизация?

Ключевое определение

Авторизация — это процесс проверки того, какие действия разрешено выполнять конкретному пользователю в системе.

Авторизация vs Аутентификация

Аутентификация — кто ты? (логин/пароль)

Авторизация — что тебе можно? (права доступа)

Авторизация всегда следует после успешной аутентификации.

Зачем нужна система ролей?

Роли позволяют гибко управлять доступом — разные пользователи получают разные возможности в зависимости от своего типа.

Безопасность

Защита критических функций от несанкционированного использования

Гибкость

Тонкая настройка прав для каждой группы пользователей

Порядок

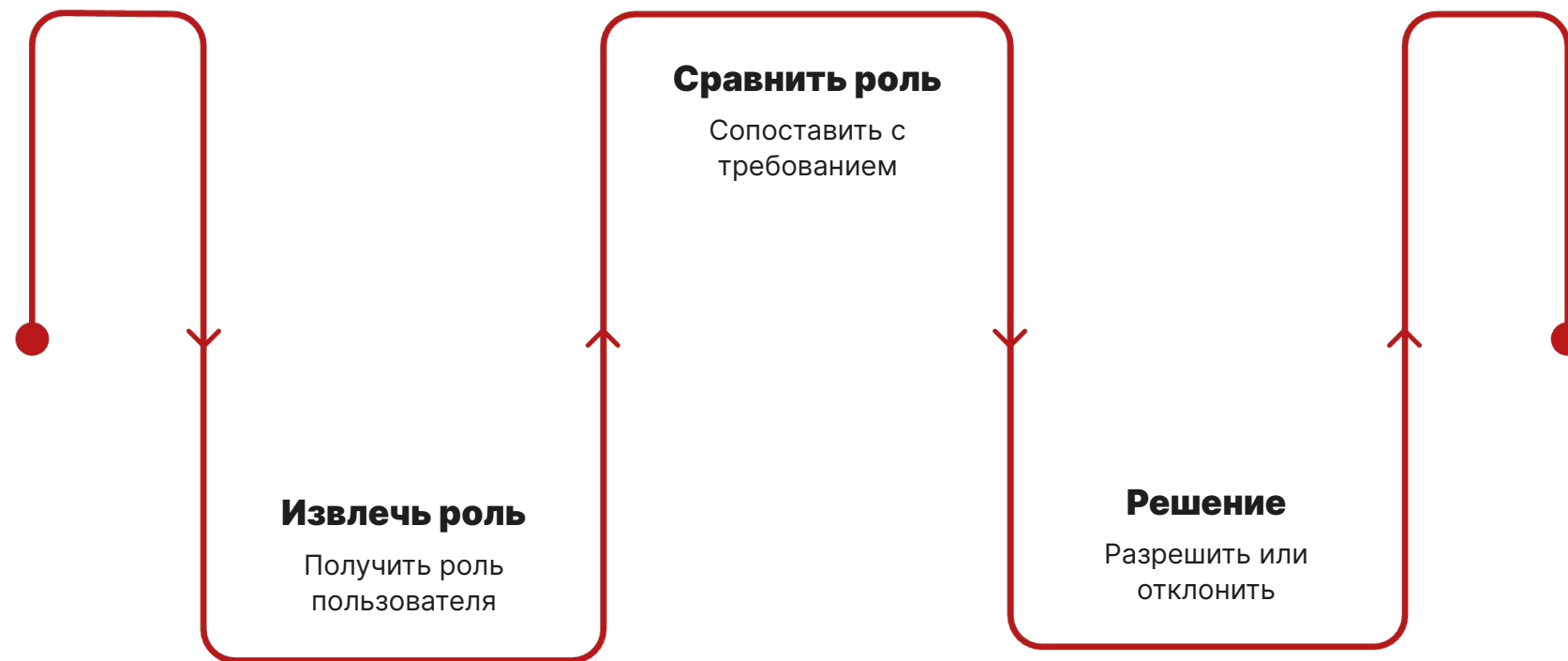
Чёткое разделение ответственности внутри системы

Примеры ролей и их права

В большинстве приложений выделяют три базовых уровня доступа:

Действие	 Пользователь	 Менеджер	 Админ
Просмотр данных			
Редактирование			
Удаление записей			
Управление ролями			

Принцип контроля доступа



Главное правило

Перед выполнением **любого защищённого действия** система обязана проверить роль пользователя.

⚠️ Никогда не доверяйте данным, пришедшим с клиента — проверяйте роль на сервере!



Практика: запрос данных через API

Получение списка продуктов

```
import axios from 'axios';

const getProducts = async
(token) => {
  const response = await
  axios.get(
    '/api/products',
    {
      headers: {
        Authorization: `Bearer
${token}`
      }
    }
  );
  return response.data;
};
```

Что здесь происходит?

- Отправляем GET-запрос к эндпоинту `/api/products`
- Передаём токен авторизации в заголовке `Authorization`
- Сервер проверяет токен и роль, затем возвращает данные

Ручная проверка роли через `if`

Простейший способ реализовать контроль доступа — использовать условные операторы для проверки роли пользователя.

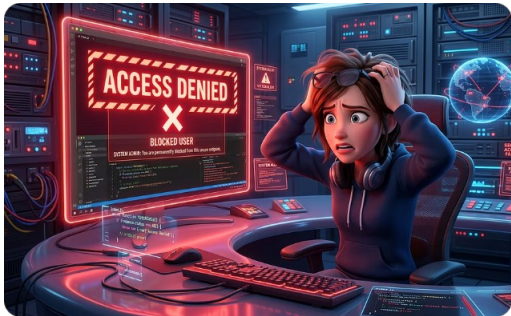
```
const handleAction = (user, action) => {  
  if (user.role === 'admin') {  
    // Администратор может всё  
    return performAction(action);  
  }  
  
  if (user.role === 'manager') {  
    // Менеджер – только редактирование  
    if (action !== 'delete') {  
      return performAction(action);  
    }  
  }  
  
  throw new Error('Доступ запрещён');  
};
```

 **Совет:** При большом количестве ролей удобнее использовать объект-словарь с разрешениями вместо цепочки `if`.

Плюсы подхода

- Простота и читаемость кода
- Легко понять даже новичку
- Быстро реализуется

Сценарий: удаление и редактирование данных



Обычный пользователь

Может только **просматривать** данные. Кнопки «Удалить» и «Изменить» скрыты или заблокированы.



Менеджер

Может **редактировать** записи, но удаление недоступно — система отклоняет запрос на сервере.



Администратор

Имеет **полный доступ**: просмотр, редактирование и удаление любых записей в системе.

Итоги занятия

Авторизация

Определяет, **что разрешено** пользователю — в отличие от аутентификации, которая устанавливает личность.

Система ролей

Роли (**user, manager, admin**) задают уровни доступа и защищают критические операции.

Практическая реализация

Axios для запросов к API + проверка роли через `if` — основа контроля доступа на клиенте и сервере.

- ✔ Система ролей — это фундамент безопасного приложения. Всегда проверяйте права на сервере!

